

AI-Powered Shiny Chatbots with ellmer and shinychat

ITEC 300 — R Package Project

Jack Markert

2026-09-15

Contents

1. Purpose	1
2. Core Functions with Example Analysis	2
3. Advantages and Limitations	7
4. Summary	8
5. References	8

1. Purpose

The `ellmer` and `shinychat` packages are a pair of R packages developed by Posit that enable developers to build AI-powered chat interfaces within Shiny applications. Together they help to bridge the gap between large language models (LLMs) and interactive R dashboards in a simple way for developers.

Ellmer provides a unified R interface to dozens of LLM providers (OpenAI, Gemini, Claude, OpenRouter, etc.) through a single consistent API. It handles streaming responses, tool/function calling (where the LLM can request execution of R functions), structured data extraction, and includes built-in provider-side tools like web search and URL fetching.

Shinychat provides the Shiny UI components for rendering the Ellmer chat interfaces. It offers `chat_ui()` to embed a chat widget, `chat_append()` to stream LLM responses in real time, and `chat_server()` for higher-level wiring with built-in support for conversation history, cancellation, file attachments, and slash commands. These functions allow ellmer LLM conversations to appear in a simple way within Shiny applications that users are familiar with.

Why Are These Packages Useful?

Before `ellmer` and `shinychat`, adding a chatbot to a Shiny app required substantial custom infrastructure such as manually handling streaming WebSocket connections, parsing provider-specific API response formats, rendering markdown in the UI, and managing conversation state. These packages collapse that work into roughly 10 lines of code. Compared to the Python ecosystem with things like StreamLit, `ellmer` and `shinychat` offer a fully native R experience. Everything runs in R and integrates directly with Shiny's reactive programming model.

Typical Use Cases

While the use cases with tailoring an LLM for individual projects is almost limitless, here are a few that I have experimented with in this project or interacted with.

- **Data exploration chat bots:** Let users ask natural language questions about a dataset
- **Dashboard assistants:** Explain visualizations or run custom analyses on demand
- **Document Q&A interfaces:** Where users upload files and ask questions about their contents
- **Interactive tutorials:** Where an AI walks users through a workflow step by step

The examples in this whitepaper come from a real implementation. Myself and team of other students from a previous semester developed a College Data Shiny Dashboard that lets users explore US college statistics such as admissions, costs, earnings, demographics, and location data for approximately 1,865 four-year institutions. This fork of that project allows users to interact with the data we have gathered with the Ellmer chatbot. The full source code is available at the accompanying GitHub repository allowing anyone to clone it and run the app themselves.

2. Core Functions with Example Analysis

The following examples are all drawn from the College Data Shiny Dashboard (`app.R`). Each demonstrates a key function from `ellmer` or `shinychat` in the context of the real application. Three examples are presented, each following the same structure: an explanation of the function, the relevant code from the dashboard, and a screenshot of the feature in action.

2.1 Example A: Chat Initialization and Web Search Grounding

`chat_google_gemini()` initializes a chat session with Google's Gemini model. The function reads your API key from the `.Renviron` `GEMINI_API_KEY` environment variable automatically. The `system_prompt` argument sets the bot's behavior. In this case, instructing it about the college dataset, available tools, and how to format responses. I have given the model two tools that it must decide to use based on the users query. Either a SQL search tool to access the data, or a web search option to find information not in the data set.

While web searching can be done with a custom tool, there is fortunately a provider-side tool provided by Gemini's `google_tool_web_search()` that automatically executes the web search on Google's servers and not in the R session. The model decides when and what to search based on the user's question, and results come back summarized for the user. No custom search API code is needed and it is one line to register.

Code from `app.R`:

```
# System prompt describing the data and available tools
system_prompt <- paste(
  "You are a helpful assistant for exploring US college data from the College Scorecard dataset.",
  "The database contains information about approximately 1,865 four-year US colleges and universities.",
  "",
  "You have two tools available:",
  "1. `query_college_data` - Execute SQL SELECT queries against the DuckDB colleges table.",
  "2. Google web search - Automatically used for questions outside the scope of the college data",
  "   (e.g., admissions events, campus life, weather, news, general information).",
  "",
  "Guidelines:",
  "- Use `query_college_data` when the user asks about colleges in the dataset.",
  "- Use web search for external/current information not in the database.",
  "- When writing SQL, always use SELECT statements only.", # This is done so that the LLM is read-only
)
```

```

"- For text searches on college names, use LIKE with % wildcards for partial matches.",
"- The table is named `colleges` (case-sensitive).",
"- Be concise and helpful. Cite sources when using web search results. No emojis",
""",
"Here is the full schema of the `colleges` table:",
college_schema_text,
sep = "\n"
)

# Initialize the Gemini chat session
chat <- chat_google_gemini(
  system_prompt = system_prompt,
  model = "gemini-3.6-flash"
)

# Register Google Search grounding
chat$register_tool(google_tool_web_search())

```

The `college_schema_text` variable referenced in the system prompt is generated by `ellmer::df_schema()`, which describes every column in the data frame (name, type, range, and unique values) so the LLM knows exactly what data is available before writing any SQL. (see Section 2.2 Example B for code where schema is generated)

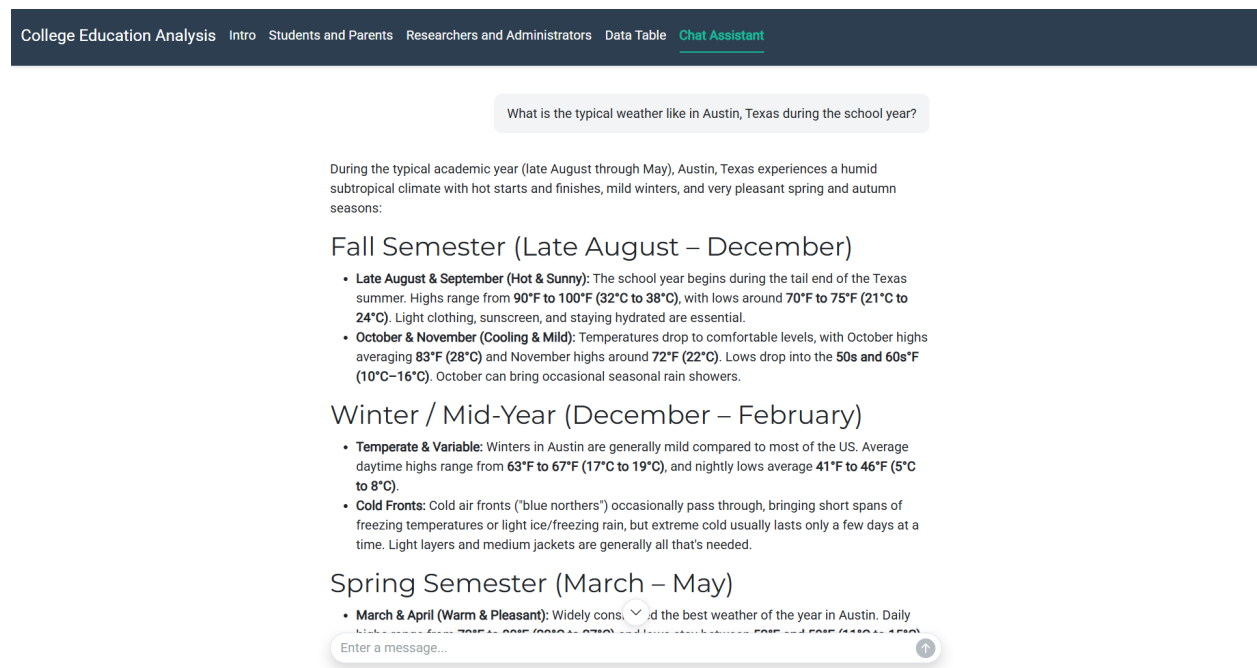


Figure 1: Image showing the Ellmer chat bot in the Shiny App replying to a message about the weather in Austin, Texas by using its web search tool.

2.2 Example B: Custom Tool Calling with `tool()`

The `tool()` function is an extremely useful function with `ellmer`. Without access to tools, it would be almost impossible for most LLMs to do the majority of tasks. The function wraps an R function with metadata

such as name, description, and typed argument specifications, so the LLM can decide when to call it and with what arguments. When the LLM requests a tool call, ellmer executes the function locally and sends the result back. The LLM never sees raw data directly so it only receives the formatted output of the local computation which it will then summarize.

In the College Data Dashboard, our data was originally a large `.csv` file. The data in this format would have been very difficult for the LLM to query for different questions. This was solved by converting the data to a duckDB database which can then be easily accessed by SQL queries which the LLM writes using a tool call. The custom tool lets the LLM write and execute the SQL queries against the duckDB database containing the full college dataset. The duckDB connection is created once at app startup:

Code from `app.R` (global):

```
# 'full_data' is the variable that is the full data of the colleges that is a .csv. Moving it to a duckDB database

# Create in-memory DuckDB database with full college data
duckdb_con <- dbConnect(duckdb::duckdb(), ":memory:")
dbWriteTable(duckdb_con, "colleges", full_data, overwrite = TRUE)

# Generate schema description for the system prompt
college_schema <- df_schema(full_data)
college_schema_text <- paste(college_schema, collapse = "\n")
```

Code from `app.R` (server) — the tool definition:

```
query_college_data <- tool(
  function(sql_query) {
    tryCatch({
      result <- DBI::dbGetQuery(duckdb_con, sql_query)
      if (nrow(result) == 0) {
        return("Query returned 0 rows. Try broadening your search criteria.") # Used if LLM search is too narrow
      }
      if (nrow(result) > 50) {
        truncated <- head(result, 50)
        out <- paste(utils::capture.output(print(truncated, max = 50)), collapse = "\n")
        return(paste0(out, "\n\n(Results truncated to first 50 of ", nrow(result), " total rows. Refine your search criteria.)")
      }
      paste(utils::capture.output(print(result, max = 50)), collapse = "\n")
    }, error = function(e) {
      paste("SQL Error:", e$message, "\nPlease check your query syntax and try again.") # LLM had syntax error
    })
  },
  name = "query_college_data",
  description = "Execute a SQL SELECT query against the `colleges` table to retrieve college data. Use standard SQL syntax.",
  arguments = list(
    sql_query = type_string("A SQL SELECT query to run against the `colleges` table. Use standard SQL syntax.")
  )
)
chat$register_tool(query_college_data)
```

Important note: The tool function runs deterministically. The same SQL query always returns the same result from the static duckDB database. The non-deterministic LLM is the router that decides what to ask and interprets the SQL query output. I point this out as this is a key aspect of the reproducibility principle I discuss later in Section 3.

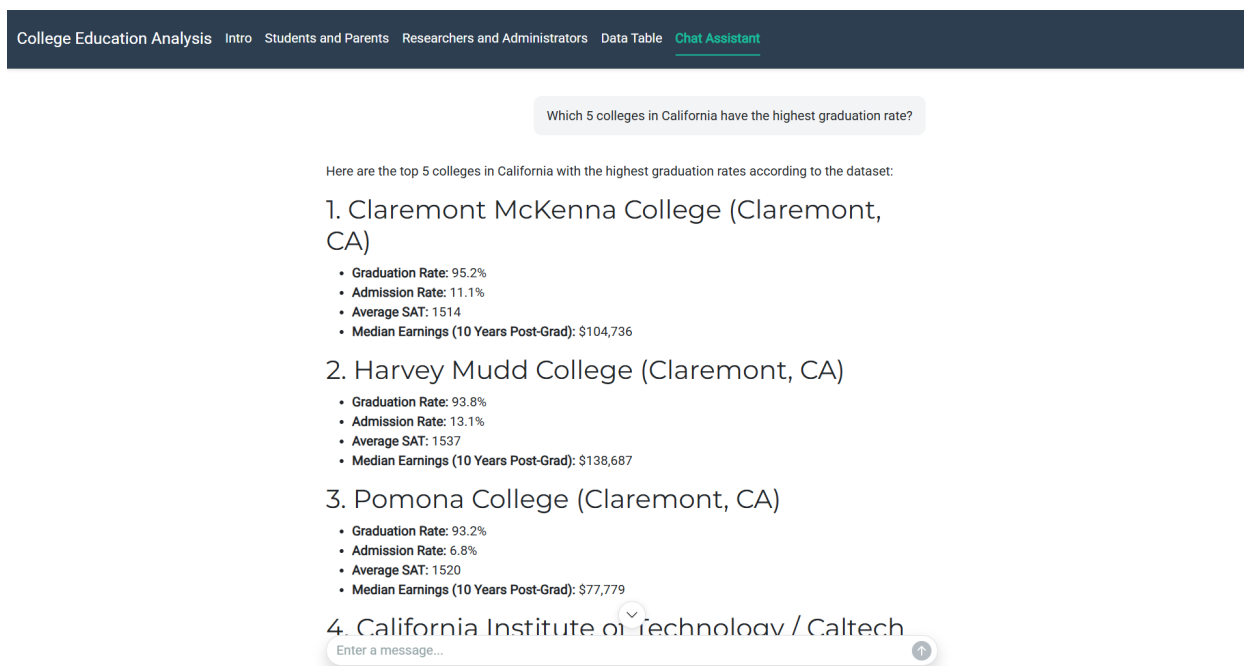


Figure 2: Image showing the Ellmer chat bot in the Shiny App replying to a message about which 5 colleges in California have the highest graduation rate. Showcasing the use of the tool call to query the data set.

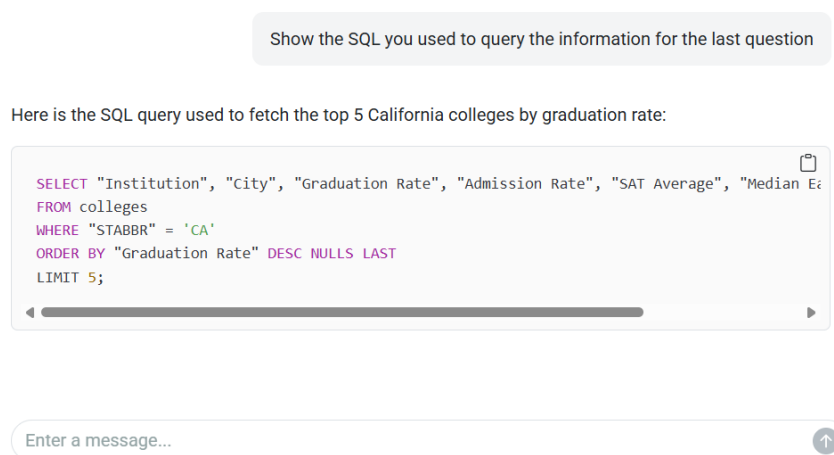


Figure 3: Image showing the Ellmer chat bot in the Shiny App replying to a message asking to show the SQL code used for the last question. The chat bot responds with the SQL code it used.

2.3 Example C: Shiny Chat UI with Streaming

Once the chat object is configured with tools, embedding it in a Shiny app is very straightforward thanks to shinychat. `chat_ui()` creates the chat widget, and `chat_append()` streams responses as they arrive from the LLM.

Code from `app.R` (UI):

```
# --- Main Tab 5: Chat Assistant ---
tabPanel(
  "Chat Assistant",
  chat_ui("college_chat", height = "calc(100vh - 150px)")
)
```

The chat UI is placed inside a `tabPanel()` within the existing `navbarPage()` layout. The height uses a `calc()` expression so it fills the viewport minus space for the navigation bar.

Code from `app.R` (server):

```
observeEvent(input$college_chat_user_input, {
  stream <- chat$stream_async(!!!input$college_chat_user_input)
  chat_append("college_chat", stream)
})
```

When a user types a message and presses Enter, `input$college_chat_user_input` runs with the message. `chat$stream_async()` sends it to Gemini and returns an asynchronous stream. As tokens arrive from the API, `chat_append()` renders them incrementally in the chat UI so the user sees the response appear word by word rather than waiting for the full answer. If the model decides to call a tool (SQL query or web search), ellmer handles the execution and the final response incorporates the tool results.

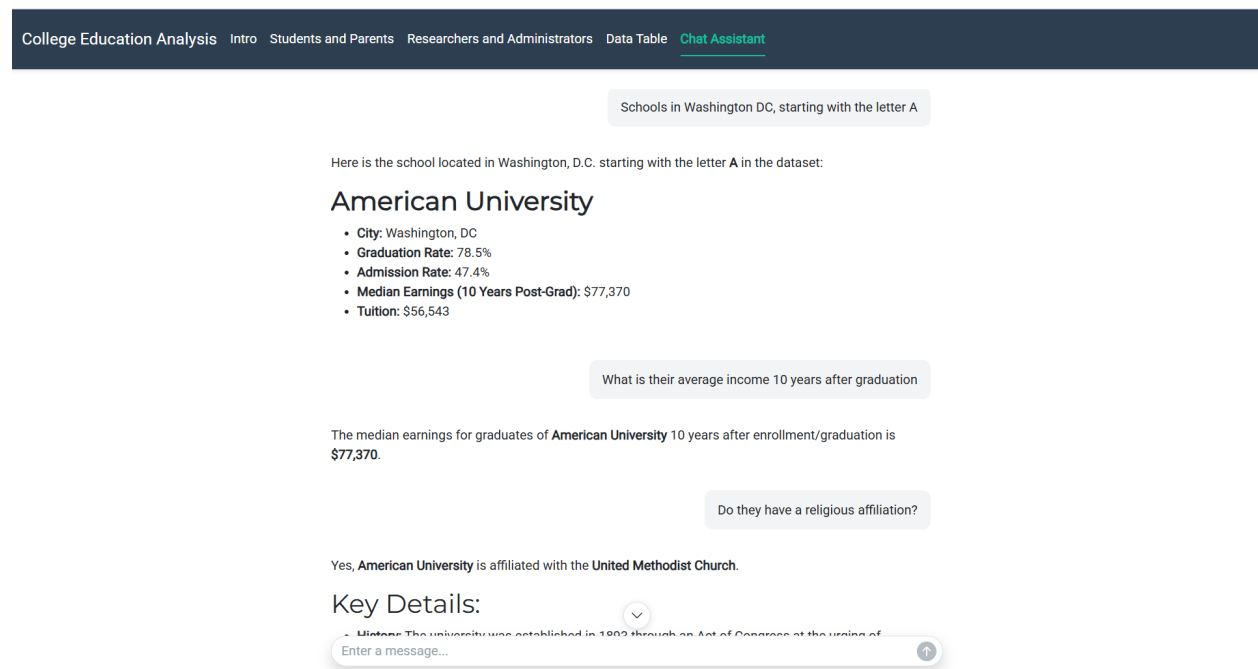


Figure 4: Image showing the Ellmer chat bot in the Shiny App replying to a messages asked by the user.

Personal Testing

The complete source code for the fork of the College Data Shiny Dashboard is available on GitHub [here](#) (link). To run it:

1. Clone the repository
2. Generate a Gemini API key from Google AI Studio (free tier works fine for testing)
3. Set your Gemini API key in `.Renviron: GEMINI_API_KEY=your_key_here`
4. Install the required packages: `install.packages(c("shiny", "ellmer", "shinychat", "duckdb", "DBI", "readr", "dplyr", "leaflet", "DT", "tidyverse", "gridExtra", "bslib", "thematic"))`
5. Run `shiny::runApp()` from the project directory

Navigate to the “Chat Assistant” tab and start asking questions about colleges in the dataset.

3. Advantages and Limitations

3.1 Reproducibility

Universal API: A chat initialized with `chat_google_gemini()` can be switched to `chat_openai()` with a single line change. The same `tool()` definitions, `register_tool()` calls, and `stream_async()` patterns work identically across all supported providers. Another user can reproduce the entire workflow with their own API key regardless of which provider they use.

Deterministic tool execution: As mentioning in my note earlier, when the LLM decides to call `query_college_data("SELECT ...")` the R function executes locally with the exact same logic every time. This is a very important feature of ellmer tool calling as the data is not being manipulated and we can predict what type of queries and answers the LLM may give and receive because we know the logic of the function. However, depending on how the LLM writes the SQL query, or how it interprets the output of the SQL, the LLM’s phrasing of an answer may vary slightly user to user.

df_schema() consistency: The schema description injected into the system prompt is generated programmatically from the data frame. This eliminates human error and inconsistency in how the data is described to the LLM, thus the same data always produces the same schema string.

3.2 Computational Power (Speed)

Streaming responses: `stream_async()` delivers tokens as they are generated. Users see responses appear incrementally, rather than waiting for the entire response to complete. With `chat_append()`, this streaming is rendered directly in the Shiny UI with no manual configuration needed.

Provider-side tools: `google_tool_web_search()` runs entirely on Google’s infrastructure. The search query, result processing, and grounding all happen server-side. This offloads both computation and complexity for our chat bot.

Efficient local queries: DuckDB executes SQL against 1,865 rows in milliseconds. The performance bottleneck in this system thus comes from the standard LLM inference time (typically 1-5 seconds), not the data layer. Querying speeds would be almost impossible if the LLM had to write R code to query the original large CSV.

Caveat: LLM inference is inherently slower than traditional computation. A SQL query that takes 5ms to execute locally may take 5 seconds to surface through the LLM because the model must generate the query, wait for execution, and then generate a natural language response. This is a trade-off of the flexibility of natural language versus direct SQL speed.

3.3 Presentation Capabilities (Primary)

The primary principle these packages embody is presentation capabilities. Their entire purpose is transforming complex LLM interactions into polished, accessible user interfaces.

`ellmer` allows us to easily set up a LLM within R with minimal code. The ability to add tools further helps to give a clean presentation with customization. The included web search tools comes default to allows for simple web search capabilities and clean results. Content streaming is abstracted behind a simple API.

`shinychat` allows to take that `ellmer` chat out of the R console and onto our shiny applications. Raw streaming API JSON is converted into styled chat bubbles with markdown rendering, syntax-highlighted code blocks, and input suggestions. A non-technical user sees a familiar LLM conversation instead of a API endpoint, JSON responses, and function tool calls.

What makes this unique is that unlike visualization packages that present data (`ggplot2` making a scatterplot, `DT` making a table), these packages present AI reasoning. The user types “*Which colleges in Texas have the best value?*” and the system:

1. Interprets the natural language intent
2. Decides whether to query the database, search the web, or both
3. Writes and executes the appropriate SQL
4. Formats the results into a coherent answer
5. Renders it as styled markdown with citations

The main point being that a user never knows about SQL, tool calling, API keys, or duckDB. The entire LLM technical pipeline is hidden behind the usually chat interface which creates a perfect presentation capability

3.4 Limitations

- **API dependency:** Requires an active internet connection and a valid API key from a supported provider.
- **Per-query costs:** While Gemini 3.6 Flash is relatively inexpensive, sustained usage in a multi-user dashboard incurs ongoing costs.
- **Package maturity:** Both packages are at version 0.5.0 as of this writing. APIs may change between versions, and breaking changes are possible.
- **R/Shiny ecosystem only:** These packages are designed exclusively for R and Shiny. The same patterns are not portable to Python, JavaScript, or other frameworks.
- **Provider-specific features:** Not all providers support all features. While the Google search tool is specific to Gemini models and something like Claude has its own web search tools, local models through something like Ollama have no built-in web search tools. Thus requiring additional setup.
- **Latency:** Each conversation turn involves at least one API round-trip (potentially more with tool calls), introducing 5-10 seconds of latency per response.

4. Summary

`Ellmer` and `shinychat` make it possible to add sophisticated AI capabilities to R Shiny dashboards with minimal code. They handle the hard problems such as provider integration, streaming, tool calling, and UI rendering so developers can easily add these increasingly popular LLM assets to their Shiny applications.

5. References

- `ellmer` package documentation: <https://ellmer.tidyverse.org/>
- `shinychat` package documentation: <https://posit-dev.github.io/shinychat/r/>

- Google Gemini API: <https://ai.google.dev/gemini-api>
- College Scorecard data: <https://collegescorecard.ed.gov/data/>
- DuckDB documentation: <https://duckdb.org/docs/>